

SparseGF2

A Mathematical Specification

August 11, 2026

Abstract

This document specifies, in complete mathematical detail, the algorithms implemented in the SparseGF2 stabilizer simulator and its circuits layer. SparseGF2 is a *phase-free* sparse stabilizer simulator over $\mathbb{F}_2$: it tracks only the symplectic part of an Aaronson–Gottesman tableau, stored sparsely, and generates and samples the symplectic group $\text{Sp}(2n, \mathbb{F}_2)$ natively. Part I covers the simulator core (representation, gates, measurement, $\mathbb{F}_2$ linear algebra, the symplectic group, observables). Part II covers the circuits layer (graph-defined monitored Clifford circuits). Part III records implemented extensions and the smaller set of capabilities that remain future work.

Contents

I	The Simulator Core	3
1	Stabilizer formalism	3
1.1	The Pauli group	3
1.2	Stabilizer states	3
1.3	The Aaronson–Gottesman tableau	3
2	The symplectic representation over $\mathbb{F}_2$	4
2.1	The phase-free coordinate map	4
2.2	The symplectic form and commutation	4
2.3	Cliffords as symplectic matrices	4
2.4	The phase-free contract	5
3	The sparse representation	5
3.1	The hybrid dense representation	5
4	Gate application	6
4.1	Symplectic action via a lookup table	6
4.2	Named gates	6
5	Measurement	6
5.1	The projection	6
5.2	The Aaronson–Gottesman update	7
5.3	Minimum-weight pivot	8
5.4	Outcome, other bases, and resets	8

6	Linear algebra over $\mathbb{F}_2$	8
6.1	Reduced row-echelon form	8
6.2	Kernel basis	8
6.3	Canonical form	8
7	The symplectic group $\text{Sp}(2n, \mathbb{F}_2)$	9
7.1	Symplectic complements	9
7.2	Order	9
7.3	Enumeration	10
7.4	Why uniform on Sp is “random Clifford”	10
7.5	Uniform sampling for arbitrary n	10
8	Observables	11
8.1	Entanglement entropy	11
8.2	Mutual and tripartite mutual information	12
8.3	Code dimension	12
8.4	Weight spectrum	12
II	The Circuits Layer	12
9	Graph-defined monitored circuits	12
9.1	Graphs, matchings, and 1-factorizations	12
9.2	Gating	13
9.3	Measurement process: candidates and firing	13
9.4	Depth normalization	14
9.5	Expected gate-to-measurement ratio	14
9.6	Time series and checkpoints	14
10	Physics pictures and order parameters	14
11	Reproducibility	15
III	Implementation status and remaining roadmap	15
12	Bravyi–Maslov $O(n^2)$ sampling	15
13	Koenig–Smolin integer indexing	15
14	Batched simulation	16
15	Additional graph families	16
16	Additional measurement modes	16
17	Adaptive depth and time series	16
18	Analysis and finite-size scaling	16

Part I

The Simulator Core

1 Stabilizer formalism

1.1 The Pauli group

For a single qubit, the Pauli matrices are I, X, Y, Z with $Y = iXZ$. The n -qubit Pauli group $\mathcal{P}_n$ consists of tensor products of these with an overall phase in $\{\pm 1, \pm i\}$. Every $P \in \mathcal{P}_n$ can be written

$$P = i^\delta \bigotimes_{q=0}^{n-1} X_q^{x_q} Z_q^{z_q}, \quad x_q, z_q \in \{0, 1\}, \quad \delta \in \mathbb{Z}_4, \quad (1)$$

so $I \leftrightarrow (0, 0)$, $X \leftrightarrow (1, 0)$, $Z \leftrightarrow (0, 1)$, $Y \leftrightarrow (1, 1)$. Two Paulis commute or anticommute; they anticommute iff the number of qubits on which their single-qubit factors anticommute is odd.

1.2 Stabilizer states

Definition 1.1. A *stabilizer group* on n qubits is an abelian subgroup $\mathcal{S} \leq \mathcal{P}_n$ with $-I \notin \mathcal{S}$ and $|\mathcal{S}| = 2^n$. Its unique simultaneous $+1$ eigenstate $|\psi\rangle$ (a one-dimensional subspace) is the associated *stabilizer state*.

A stabilizer group is fixed by n independent generators. A *Clifford* unitary C is one for which $C\mathcal{P}_n C^\dagger = \mathcal{P}_n$; Cliffords map stabilizer states to stabilizer states, acting on generators by conjugation.

1.3 The Aaronson–Gottesman tableau

The Aaronson–Gottesman (A&G) representation [1] tracks $2n$ generators: n *stabilizers* $S_0, \dots, S_{n-1}$ generating $\mathcal{S}$, and n *destabilizers* $D_0, \dots, D_{n-1}$ chosen so that the full set $\{D_i, S_i\}$ generates $\mathcal{P}_n$ (modulo phase) and satisfies the canonical (anti)commutation relations

$$[D_i, D_j] = [S_i, S_j] = 0, \quad D_i S_j = (-1)^{\delta_{ij}} S_j D_i. \quad (2)$$

That is, D_i anticommutes with S_i and commutes with every other generator. The destabilizers carry no physical information about $|\psi\rangle$; they are bookkeeping that makes Clifford updates and measurements $O(n)$ per generator.

SparseGF2 orders the $2n$ generators as rows $0, \dots, n-1$ (destabilizers) and $n, \dots, 2n-1$ (stabilizers). The initial state $|0^n\rangle$ has $S_i = Z_i$, $D_i = X_i$.

2 The symplectic representation over $\mathbb{F}_2$

2.1 The phase-free coordinate map

Dropping the phase i^δ in (1) sends a Pauli to its *symplectic coordinates* $(x|z) = (x_0, \dots, x_{n-1}, z_0, \dots, z_{n-1}) \in \mathbb{F}_2^{2n}$. SparseGF2 works throughout in the ordered basis $(X_0, \dots, X_{n-1}, Z_0, \dots, Z_{n-1})$, so a Pauli is a length- $2n$ bit row, the X -block first. Stacking the $2n$ generators gives the *symplectic tableau* $T \in \mathbb{F}_2^{2n \times 2n}$, written $[X|Z]$; rows $0..n-1$ are destabilizers, rows $n..2n-1$ stabilizers.

Example 2.1. $|0^n\rangle$ has destabilizers X_i and stabilizers Z_i , so its tableau is the $2n \times 2n$ identity: the top-left $n \times n$ block (X-bits of the X_i) and the bottom-right block (Z-bits of the Z_i) are I_n , the rest 0.

2.2 The symplectic form and commutation

Definition 2.2. The *symplectic form* on $\mathbb{F}_2^{2n}$ in the X - Z block basis is

$$\Omega = \begin{pmatrix} 0_n & I_n \\ I_n & 0_n \end{pmatrix} \in \mathbb{F}_2^{2n \times 2n}, \quad (3)$$

and the *symplectic inner product* of $u = (u_x|u_z)$, $v = (v_x|v_z)$ is

$$\langle u, v \rangle_\Omega = u \Omega v^\top = u_x \cdot v_z + u_z \cdot v_x \pmod{2}. \quad (4)$$

(Over $\mathbb{F}_2$, $-1 = +1$, so the usual $\begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}$ coincides with Ω .)

Proposition 2.3. Two Paulis with coordinates u, v commute iff $\langle u, v \rangle_\Omega = 0$ and anticommute iff $\langle u, v \rangle_\Omega = 1$.

Proof. On qubit q the factors $X^{u_{x,q}} Z^{u_{z,q}}$ and $X^{v_{x,q}} Z^{v_{z,q}}$ anticommute iff $u_{x,q} v_{z,q} + u_{z,q} v_{x,q} = 1$ (using $XZ = -ZX$). The global sign is the product over q , i.e. (-1) to the sum $\sum_q (u_{x,q} v_{z,q} + u_{z,q} v_{x,q}) = \langle u, v \rangle_\Omega$. $\square$

The canonical relations (2) state that the tableau rows form a *symplectic basis*: with e_i the destabilizer rows and f_i the stabilizer rows, $\langle e_i, f_j \rangle_\Omega = \delta_{ij}$ and $\langle e_i, e_j \rangle_\Omega = \langle f_i, f_j \rangle_\Omega = 0$.

2.3 Cliffords as symplectic matrices

A phase-free Clifford acts on Pauli coordinates by a linear map. We use the *row-vector convention*: a Pauli row $(x|z)$ evolves under a gate with matrix S as

$$(x'|z') = (x|z) S \pmod{2}. \quad (5)$$

The rows of S are the images of the basis Paulis, in the order $(X_0, \dots, Z_{n-1})$.

Definition 2.4. $S \in \mathbb{F}_2^{2n \times 2n}$ is *symplectic* if it preserves Ω :

$$S \Omega S^\top \equiv \Omega \pmod{2}. \quad (6)$$

The set of such S is the group $\text{Sp}(2n, \mathbb{F}_2)$.

Proposition 2.5. A linear map (5) sends Paulis to Paulis preserving all commutation relations iff $S \in \text{Sp}(2n, \mathbb{F}_2)$. Hence phase-free Cliffords are exactly $\text{Sp}(2n, \mathbb{F}_2)$.

Proof. $\langle uS, vS \rangle_\Omega = uS \Omega S^\top v^\top$ equals $\langle u, v \rangle_\Omega = u \Omega v^\top$ for all u, v iff $S \Omega S^\top = \Omega$. $\square$

SparseGF2 validates (6) on every user-supplied gate matrix.

2.4 The phase-free contract

SparseGF2 stores $[X \mid Z]$ only; it discards the phase bits. Two consequences:

1. **What is exact.** Every quantity that depends only on the $\mathbb{F}_2$ *row span* of the stabilizer tableau is computed exactly: $\mathbb{F}_2$ -rank, entanglement entropy, mutual information, code dimension, and the stabilizer weight spectrum (Section 8). Gates differing only by phase (e.g. S vs $S^\dagger$, a Pauli vs I) collapse to the same symplectic action.
2. **What is lost.** Signed Pauli expectations $\langle \psi | P | \psi \rangle \in \{\pm 1\}$ are unavailable, because the distinguishing sign lives in the discarded phase. A deterministic Z_q measurement of a $-Z_q$ -stabilized state physically yields 1, but SparseGF2 returns 0 (Section 5).

Definition 2.6 (Parity contract). The correctness guarantee is *stabilizer-subspace RREF parity*: at every step of every circuit, the reduced row-echelon form over $\mathbb{F}_2$ of the stabilizer block (rows $n..2n-1$) of $[X \mid Z]$ equals Stim's. The test suite enforces this step by step.

3 The sparse representation

The tableau is stored in four redundant structures, all kept consistent so the hot paths (gates, measurement) touch only the generators they must.

Definition 3.1 (Data structures).

PLT (Pauli lookup table)

A dense $2n \times n$ array `plt[r, q]` holding the 2-bit code $xz = (x \ll 1) | z$ for generator r on qubit q : $I=0, Z=1, X=2, Y=3$. Gives $O(1)$ random reads of any Pauli letter.

Support lists

For each generator r , the list `supp_q[r, ·]` of qubits on which r is non-identity, with length `supp_len[r]` and an inverse position map `supp_pos` enabling $O(1)$ removal (swap-with-last). Lets one iterate a generator in $O(\text{wt}(r))$.

Inverted index inv

For each qubit q , the list of generators with support on q , with length and position maps. Gate kernels iterate this so a gate on qubits $\{q_i, q_j\}$ touches only the affected rows.

Inverted index inv_x

For each qubit q , the list of generators whose X -bit on q is set. These are exactly the generators anticommuting with Z_q ; the measurement kernel uses this to find anticommuters in $O(1)$ per element.

The *weight* of a generator is $\text{wt}(r) = \text{supp_len}[r]$, the number of non-identity Paulis. Sparsity of the tableau (wt small) is what makes both gates and measurements cheap; see Sections 5 and 8.

3.1 The hybrid dense representation

The sparse representation wins precisely when the tableau is sparse. In the volume-law regime the generators delocalize ($\text{wt}(r) = O(n)$), and then the inverted-index bookkeeping (Definition above) costs more than it saves: each gate touches $O(n)$ rows and pays the index-maintenance overhead

on every one. There a plain bit-packed tableau is faster, because a gate is a fixed handful of word operations per row with no bookkeeping.

The *hybrid* mode (`SparseGF2(n, hybrid=True)`) carries a second, bit-packed mirror of the tableau, two $2n \times \lceil n/64 \rceil$ arrays of `uint64`, `x_packed` and `z_packed`, with qubit q at word $q \gg 6$, bit $q \& 63$, and switches between the two representations based on a density estimate. Let

$$\bar{a} = \frac{1}{n} \sum_{r=0}^{2n-1} \text{wt}(r)$$

be the average column weight (the mean number of generators touching a qubit). The simulator runs the sparse kernels while $\bar{a}$ is small and switches to the dense bit-packed kernels (a phase-free Aaronson–Gottesman update applied to the packed rows, reusing the same symplectic lookup tables of Section 4) once $\bar{a}$ exceeds a threshold $\tau = \max(n/4, 16)$, switching back below $\tau/2$. The $2\times$ hysteresis band, together with evaluating the test only every $\max(n/2, 32)$ operations, prevents thrashing near the crossover. The two representations describe the *same stabilizer group* at all times, the dense measurement chooses a different (equally valid) generating set than the sparse minimum-weight pivot (Section 5.3), but every gauge-invariant observable (Section 8) is identical. The dense path recovers Stim-like performance in the volume-law regime while keeping the sparse advantage in the area-law regime, from one object.

4 Gate application

4.1 Symplectic action via a lookup table

By (5), a k -qubit gate S ($k \in \{1, 2\}$) acts on a generator only through that generator’s $2k$ coordinate bits on the gated qubits; all other coordinates are unchanged. `SparseGF2` precomputes, once per distinct S , a *lookup table* (LUT) mapping each of the 4^k possible local Pauli codes to its image:

$$\text{LUT}_S: \{0, 1, 2, 3\}^k \longrightarrow \{0, 1, 2, 3\}^k, \quad (\text{local } xz \text{ in}) \mapsto (\text{local } xz \text{ out}), \quad (7)$$

i.e. 4 entries for a single-qubit gate, 16 for a two-qubit gate. Applying S to the whole state then iterates the generators with support on the gated qubits (via `inv`), reads their local code from the PLT, and rewrites it through the LUT, patching the support lists and inverted indices in $O(1)$ per affected bit. Cost is $O(|\bigcup_q \text{inv}[q]|)$, independent of n when the gate is local and the tableau sparse.

4.2 Named gates

The standard generators are the symplectic matrices (row-vector convention)

$$H: \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad S: \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \sqrt{X}: \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

on a single qubit (acting on $(x|z)$), and $CX, CZ, \text{SWAP} \in \text{Sp}(4, \mathbb{F}_2)$ on two. Arbitrary $S \in \text{Sp}(2k, \mathbb{F}_2)$ are accepted via `apply_gate_1q/apply_gate_2q`, with (6) checked on the first use of each matrix.

5 Measurement

5.1 The projection

A Z_q measurement projects onto an eigenspace of Z_q . In the stabilizer formalism there are two cases, determined by whether Z_q is already “known.”

Proposition 5.1 (Determinism condition). *A Z_q measurement is deterministic (outcome fixed by the state) iff $Z_q \in \langle S_0, \dots, S_{n-1} \rangle$, equivalently iff no stabilizer row anticommutes with Z_q : no row $r \geq n$ has its X -bit on q set. Otherwise the outcome is a fair coin and the state is projected.*

Proof. The outcome is deterministic iff $\pm Z_q \in \mathcal{S}$. As an n -dimensional isotropic subspace of $\mathbb{F}_2^{2n}$, $\mathcal{S}$ is maximal isotropic and so equals its own symplectic complement; hence $\pm Z_q \in \mathcal{S}$ iff Z_q commutes with all of $\mathcal{S}$, i.e. (commutation Proposition) iff no stabilizer generator has its X -bit on q . A destabilizer's X -bit on q reflects only the auxiliary pairing (2), not membership in $\mathcal{S}$, so it is irrelevant. $\square$

SparseGF2 reads the anticommuters directly from `inv_x[q]` and checks whether any has row index $\geq n$.

5.2 The Aaronson–Gottesman update

In the non-deterministic case, let $\mathcal{A} = \{r : x_{r,q} = 1\}$ be the generators anticommuting with Z_q (from `inv_x[q]`), and pick a *pivot* stabilizer $p \in \mathcal{A}$, $p \geq n$. The phase-free update is:

1. **Decouple.** For every other $r \in \mathcal{A}$, replace $T_r \leftarrow T_r \oplus T_p$ (bitwise XOR of rows). Afterwards p is the unique generator anticommuting with Z_q .
2. **Re-pair.** Copy the old pivot row into the destabilizer slot $p - n$: $T_{p-n} \leftarrow T_p$. This preserves the canonical pairing (2) (the new stabilizer Z_q will anticommute with exactly the old S_p , now sitting at $p - n$).
3. **Install.** Overwrite $T_p \leftarrow Z_q$.

(The XOR in Step 1 is skipped for $r = p - n$, since Step 2 overwrites that slot.) In the deterministic case the symplectic tableau is left *unchanged*: Stim would only flip a sign bit, which SparseGF2 does not store, so this matches Stim's symplectic exactly.

Proposition 5.2. *The update of Section 5.2 produces the unique symplectic tableau of the post-measurement state, independent of the pivot choice and of the (phase-free) outcome.*

Proof. Let $\mathcal{S}$ be the pre-measurement stabilizer group and $\mathcal{S}^0 = \{g \in \mathcal{S} : [g, Z_q] = 0\}$ its subgroup commuting with Z_q ; in the non-deterministic case some stabilizer anticommutes with Z_q , so $\mathcal{S}^0$ has index 2 and dimension $n - 1$. The projection onto the Z_q -eigenspace is the pure state stabilized by $\langle Z_q, \mathcal{S}^0 \rangle$.

In coordinates, $\mathcal{A} = \{r : x_{r,q} = 1\}$ are the generators anticommuting with Z_q and $p \in \mathcal{A}$ is the stabilizer pivot. *Step 1:* for each stabilizer $r \in \mathcal{A} \setminus \{p\}$, the row $T_r \oplus T_p$ has q -th X -bit $1 \oplus 1 = 0$, hence commutes with Z_q ; adding the fixed row T_p to others is a change of generating set, so the stabilizer span is preserved. Afterwards T_p is the *unique* anticommuting stabilizer, and the remaining $n - 1$ stabilizer rows are independent elements of the $(n - 1)$ -dimensional $\mathcal{S}^0$, hence a basis of it. *Step 3:* replacing T_p by Z_q makes the stabilizer rows generate $\langle Z_q, \mathcal{S}^0 \rangle$, exactly the projected state.

Step 2 restores the canonical relations (2). Before the update D_{p-n} was the unique destabilizer anticommuting with $S_p = T_p$. Setting $D_{p-n} \leftarrow T_p$ (the old S_p) gives a destabilizer that anticommutes with the new stabilizer Z_q at slot p (the old $S_p \in \mathcal{A}$ had x -bit on q) and commutes with every other stabilizer (those were made to commute with Z_q in Step 1 and already commuted with S_p). Every other destabilizer is untouched and keeps its relations; hence (2) holds. (The XOR of T_p into slot $p - n$ is skipped in Step 1 precisely because Step 2 overwrites it.)

Finally, different pivots yield different generating sets of the same group $\langle Z_q, S^0 \rangle$, i.e. different representatives of one row span; the RREF (`canonical_form`), and thus the recorded symplectic state, is identical, independent of the pivot and of the discarded outcome sign. $\square$

5.3 Minimum-weight pivot

When several stabilizers anticommute, `SparseGF2` selects the *minimum-weight* pivot, $p = \arg \min_{r \in \mathcal{A}, r \geq n} \text{wt}(r)$ (rather than the first). The decoupling XOR in Step 1 adds the pivot's support to every other anticommuter, so a light pivot keeps the tableau sparse, lowering the cost of *future* gates and measurements. The choice is invisible to all observables (it changes only the representative, not the row span).

5.4 Outcome, other bases, and resets

For a non-deterministic measurement `SparseGF2` returns a fair phase-free coin from the instance RNG; for a deterministic one it returns 0 (see Section 2.4). Measurements in the X and Y bases are conjugations of Z : `measure_x` = $H \text{measure}_z H$ and `measure_y` = $SH \text{measure}_z HS$ (phase-free, $S = S^\dagger$). Resets `reset_z/x/y` apply the same projection and discard the bit; they project onto *some* eigenstate of the basis operator (the eigenvalue is a sign-bit fact not tracked).

6 Linear algebra over $\mathbb{F}_2$

Two primitives, shared across the simulator, with bit-identical JIT and pure-Python paths.

6.1 Reduced row-echelon form

`gf2_rref(A)` reduces $A \in \mathbb{F}_2^{m \times k}$ to RREF by Gaussian elimination: scanning columns left to right, it finds a pivot in the current column at or below the pivot row, swaps it up, and XORs that row into every other row with a 1 in the pivot column. Rank is the number of nonzero rows of the RREF, $\text{rank}_{\mathbb{F}_2}(A) \in [0, \min(m, k)]$.

6.2 Kernel basis

`gf2_kernel_basis(M)` returns a basis of the right kernel $\ker M = \{v \in \mathbb{F}_2^k : Mv^\top = 0\}$. After reducing M to RREF with pivot columns $\{p_r\}$, each *free column* f (non-pivot) yields one basis vector v with $v[f] = 1$ and $v[p_r] = A[r, f]$ for every pivot row r (all other entries 0). This gives $\dim \ker M = k - \text{rank } M$ vectors. The empty matrix ($m = 0$) returns the $k \times k$ identity. This primitive drives the symplectic sampler (Section 7.5).

6.3 Canonical form

`canonical_form()` is the RREF of the stabilizer block (rows $n..2n-1$) of $[X \mid Z]$, the canonical representative of the stabilizer *subspace*, used for state-equality testing and the parity contract of Section 2.4.

7 The symplectic group $\text{Sp}(2n, \mathbb{F}_2)$

The simulator generates and samples $\text{Sp}(2n, \mathbb{F}_2)$ natively, with no runtime dependence on Stim.

7.1 Symplectic complements

Definition 7.1. For a subspace $U \leq \mathbb{F}_2^{2n}$, the *symplectic complement* is $U^\perp = \{w \in \mathbb{F}_2^{2n} : \langle u, w \rangle_\Omega = 0 \text{ for all } u \in U\}$. A subspace W is *non-degenerate* if the restriction of Ω to W has trivial radical: no nonzero $w \in W$ has $\langle w, w' \rangle_\Omega = 0$ for all $w' \in W$.

Lemma 7.2. *The form Ω is non-degenerate on $\mathbb{F}_2^{2n}$ (indeed $\Omega^2 = I$). If U is a non-degenerate subspace then $U^\perp$ is non-degenerate, $\dim U + \dim U^\perp = 2n$, and $\mathbb{F}_2^{2n} = U \oplus U^\perp$. Consequently the span of k symplectic pairs (X'_j, Z'_j) (with $\langle X'_j, Z'_j \rangle_\Omega = 1$ and all cross-products 0) is non-degenerate of dimension $2k$, and its complement has dimension $2(n - k)$.*

Proof. The map $\phi_U : \mathbb{F}_2^{2n} \rightarrow U^*$, $w \mapsto (u \mapsto \langle u, w \rangle_\Omega)$ is linear with kernel $U^\perp$, so $\dim U^\perp = 2n - \dim \text{im } \phi_U$. When U is non-degenerate, $\Omega|_U$ is an isomorphism $U \rightarrow U^*$, so ϕ_U is surjective and $\dim U^\perp = 2n - \dim U$; moreover $U \cap U^\perp = \text{rad}(U) = \{0\}$, whence $\mathbb{F}_2^{2n} = U \oplus U^\perp$, and applying the same to $U^\perp$ shows it is non-degenerate. A single pair spans a non-degenerate 2-plane (its Gram matrix is $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, invertible); an orthogonal direct sum of such planes is non-degenerate of dimension $2k$. $\square$

7.2 Order

Theorem 7.3.

$$|\text{Sp}(2n, \mathbb{F}_2)| = \prod_{i=1}^n (2^{2(n-i+1)} - 1) 2^{2(n-i+1)-1} = 2^{n^2} \prod_{i=1}^n (2^{2i} - 1).$$

In particular $|\text{Sp}(2, \mathbb{F}_2)| = 6$, $|\text{Sp}(4, \mathbb{F}_2)| = 720$, $|\text{Sp}(6, \mathbb{F}_2)| = 1,451,520$.

Proof. The rows of any $S \in \text{Sp}(2n, \mathbb{F}_2)$ form an ordered *symplectic basis* $(X'_1, Z'_1, \dots, X'_n, Z'_n)$: condition (6) is exactly the statement that $\langle X'_i, Z'_j \rangle_\Omega = \delta_{ij}$ and $\langle X'_i, X'_j \rangle_\Omega = \langle Z'_i, Z'_j \rangle_\Omega = 0$, since $S\Omega S^\top = \Omega$ records all pairwise symplectic products of the rows. Conversely every ordered symplectic basis is the row set of a unique such S . So $|\text{Sp}(2n, \mathbb{F}_2)|$ is the number of ordered symplectic bases, which we count by building one greedily.

Suppose (X'_j, Z'_j) are chosen for $j < i$ and let U_{i-1} be their span. By Lemma 7.2, U_{i-1} is non-degenerate of dimension $2(i-1)$ and its complement $W_i = U_{i-1}^\perp$ is non-degenerate of dimension $2(n-i+1)$; the next pair must lie in W_i .

- X'_i : any nonzero vector of W_i , giving $|W_i| - 1 = 2^{2(n-i+1)} - 1$ choices.
- Z'_i : any $w \in W_i$ with $\langle X'_i, w \rangle_\Omega = 1$. The functional $\ell : W_i \rightarrow \mathbb{F}_2$, $\ell(w) = \langle X'_i, w \rangle_\Omega$, is nonzero: $X'_i \neq 0$ and W_i is non-degenerate, so some $w \in W_i$ has $\langle X'_i, w \rangle_\Omega = 1$. A nonzero $\mathbb{F}_2$ -linear functional has a kernel of codimension 1, so $\ell^{-1}(1)$ is a coset of that kernel of size $\frac{1}{2}|W_i| = 2^{2(n-i+1)-1}$.

(Any such (X'_i, Z'_i) spans a non-degenerate 2-plane in W_i , so the induction hypothesis is maintained.) Multiplying the per-step counts over $i = 1, \dots, n$ gives the first form; substituting $j = n - i + 1$ and pulling out $\prod_i 2^{2j-1} = 2^{\sum_{j=1}^n (2j-1)} = 2^{n^2}$ gives the second. $\square$

7.3 Enumeration

For $n \leq 3$, `enumerate_symplectic_group`(n) realizes the construction of Theorem 7.3 recursively: at depth i it forms W_i by filtering $\mathbb{F}_2^{2n}$ to vectors v with $\langle c, v \rangle_\Omega = 0$ for every previously chosen c (a linear sieve), then recurses over all valid (X'_i, Z'_i) . The $n = 2$ table of 720 matrices is cached and is the gate set of the circuits layer. The count is checked against Theorem 7.3. $n \geq 4$ is infeasible by enumeration ($\sim 4.7 \times 10^{10}$ elements).

7.4 Why uniform on Sp is “random Clifford”

The full n -qubit Clifford group (with signs) has order $|\mathcal{C}_n| = 2^{n^2+2n} \prod_{i=1}^n (4^i - 1) = 4^n |\text{Sp}(2n, \mathbb{F}_2)|$. The quotient map $\mathcal{C}_n \rightarrow \text{Sp}(2n, \mathbb{F}_2)$ (forget signs) is 4^n -to-1: each symplectic S lifts to 4^n signed Cliffords (a $\pm$ choice on each of the $2n$ generators). Hence pushing the uniform distribution on $\mathcal{C}_n$ forward to $\text{Sp}(2n, \mathbb{F}_2)$ gives the *uniform* distribution on the $|\text{Sp}|$ symplectics. For two qubits, $|\mathcal{C}_2| = 11,520 = 16 \cdot 720$. So for a phase-free simulator, “uniform random Clifford” *is* “uniform random element of $\text{Sp}(2n, \mathbb{F}_2)$,” and sampling the 720-element $\text{Sp}(4, \mathbb{F}_2)$ table uniformly is exactly correct.

7.5 Uniform sampling for arbitrary n

`random_symplectic`, through its `sampler` argument, exposes two uniform constructions. The explicit `kernel_basis` option realizes the *same* greedy construction with uniform choices at each step. It maintains the symplectic complement W_i of the previously chosen pairs as the right kernel of the constraint matrix

$$M_i = \left[[X'_{j,z} \mid X'_{j,x}]; [Z'_{j,z} \mid Z'_{j,x}] : j < i \right],$$

where the half-swap $[w_z \mid w_x]$ encodes $\langle u, w \rangle_\Omega = 0$ as the linear constraint $[w_z \mid w_x] u^\top = 0$ (cf. (4)). At step i , with $K = \text{gf2_kernel_basis}(M_i)$ a basis of W_i ($\dim W_i = 2(n - i)$ rows after i pairs; $2n$ initially):

1. X'_i : draw a uniform *nonzero* coefficient vector $c \in \mathbb{F}_2^{\dim W_i}$ and set $X'_i = cK$. This is uniform over $W_i \setminus \{0\}$.
2. Z'_i : let $\pi_k = \langle X'_i, K_k \rangle_\Omega$. Draw a uniform $c' \in \mathbb{F}_2^{\dim W_i}$; if its parity $c' \cdot \pi$ is 0, flip one bit of c' at an index where $\pi_k = 1$ (which exists since W_i is non-degenerate). Set $Z'_i = c'K$. This is uniform over $\{w \in W_i : \langle X'_i, w \rangle_\Omega = 1\}$.

The matrix S has rows $X'_1, \dots, X'_n, Z'_1, \dots, Z'_n$.

Proposition 7.4. *The procedure outputs each $S \in \text{Sp}(2n, \mathbb{F}_2)$ with equal probability.*

Proof. By Theorem 7.3 ordered symplectic bases biject with $\text{Sp}(2n, \mathbb{F}_2)$, and the construction reaches each basis by exactly one sequence of choices $(X'_i, Z'_i) \in (W_i \setminus \{0\}) \times \ell^{-1}(1)$, where $\ell(w) = \langle X'_i, w \rangle_\Omega$. It suffices that each step is uniform over its choice set; the joint distribution is then the product, hence uniform over all bases and over the group. Write $m = \dim W_i$ and let K be the basis of W_i .

Step 1 (uniform over $W_i \setminus \{0\}$). Drawing $c \in \mathbb{F}_2^m$ uniformly and rejecting $c = 0$ gives a uniform nonzero c ; since $c \mapsto cK$ is a linear isomorphism $\mathbb{F}_2^m \rightarrow W_i$, $X'_i = cK$ is uniform over $W_i \setminus \{0\}$.

Step 2 (uniform over $\ell^{-1}(1)$). Put $\pi_k = \langle X'_i, K_k \rangle_\Omega$, so $\langle X'_i, c'K \rangle_\Omega = c' \cdot \pi$; $\pi \neq 0$ because $\ell \neq 0$. The procedure draws $c' \in \mathbb{F}_2^m$ uniformly and, if $c' \cdot \pi = 0$, picks j uniformly from $J = \{k : \pi_k = 1\}$

($a := |J| \geq 1$) and flips c'_j (which sets the parity to 1). Fix a target t with $t \cdot \pi = 1$. It is produced (i) directly, when the draw equals t , with probability 2^{-m} ; or (ii) by correction, when the draw is $t \oplus e_j$ for some $j \in J$ (which has even parity) and index j is then selected, with probability $\sum_{j \in J} 2^{-m} \cdot a^{-1} = 2^{-m}$. So $\Pr[\text{output} = t] = 2 \cdot 2^{-m} = 2^{-(m-1)}$, uniform over the 2^{m-1} -element coset $\ell^{-1}(1)$. As $c' \mapsto c'K$ maps this coset bijectively onto $\ell^{-1}(1)$, Z'_i is uniform there. $\square$

The kernel-basis construction costs $O(n^3)$ bit-operations (a kernel computation per step). For $n = 2$ the routine short-circuits to a uniform draw from the cached $\text{Sp}(4, \mathbb{F}_2)$ table so the RNG state advances identically across entry points.

Remark 7.5. The `bravyi_maslov` option implements the canonical decomposition $C = F_1 H S F_2$ with a Mallows-distributed permutation [3]. It requires $O(n^2)$ logical bit operations (dense matrix products add implementation overhead) and is the default selected by `sampler=auto` for $n \neq 2$. The kernel-basis method remains available as the simpler independently verified construction; both distributions are uniform but consume different RNG draws (Section 12).

8 Observables

All observables are exact functions of the $\mathbb{F}_2$ row span (Section 2.4); none require phase information.

8.1 Entanglement entropy

Theorem 8.1 (Fattal–Cubitt–Yamamoto–Bravyi–Chuang [2]). *For a stabilizer state with stabilizer generators forming the matrix $G \in \mathbb{F}_2^{n \times 2n}$ (rows $n..2n-1$ of $[X \mid Z]$), and a subsystem $A \subseteq \{0, \dots, n-1\}$, the entanglement entropy (in ebits, units of $\log_2$) is*

$$S(A) = \text{rank}_{\mathbb{F}_2}(G|_A) - |A|, \quad (8)$$

where $G|_A \in \mathbb{F}_2^{n \times 2|A|}$ is G restricted to the columns A (the X -bits) and $A+n$ (the Z -bits) of the qubits in A .

Proof. For a region B , let $\mathcal{S}_B = \{g \in \mathcal{S} : \text{supp}(g) \subseteq B\}$ be the subgroup of stabilizers acting as identity outside B , and $d_B = \dim_{\mathbb{F}_2} \mathcal{S}_B$ its number of independent generators. A standard computation gives the reduced density operator of a stabilizer state on B as the uniform mixture $\rho_B = 2^{-|B|} \sum_{g \in \mathcal{S}_B} \pm g|_B$, which is $2^{-(|B|-d_B)}$ times a projector of rank $2^{|B|-d_B}$; hence $S(B) = |B| - d_B$. (The signs select *which* eigenstate, not the entropy, so the phase-free coordinates suffice.) Writing $g = \prod_i g_i^{c_i}$, we have $g \in \mathcal{S}_{\bar{A}}$ iff its A -columns vanish, i.e. iff c lies in the kernel of the restriction $c \mapsto (cG)|_A$; therefore $d_{\bar{A}} = n - \text{rank}_{\mathbb{F}_2}(G|_A)$. For a pure state $S(A) = S(\bar{A})$, so

$$S(A) = S(\bar{A}) = |\bar{A}| - d_{\bar{A}} = (n - |A|) - (n - \text{rank}_{\mathbb{F}_2}(G|_A)) = \text{rank}_{\mathbb{F}_2}(G|_A) - |A|. \quad \square$$

The bounds $0 \leq S(A) \leq \min(|A|, n - |A|)$ follow: $\mathcal{S}_A$ is an abelian (isotropic) subgroup of the $2|A|$ -dimensional symplectic space on A , so $d_A \leq |A|$ and $S(A) = |A| - d_A \geq 0$; and $S(A) = S(\bar{A}) = |\bar{A}| - d_{\bar{A}} \leq n - |A|$. `SparseGF2` raises if this contract is violated (a corruption check), and implements (8) via `gf2_rank` on the stabilizer-restricted block, avoiding the full $2n \times 2n$ allocation.

Remark 8.2. The cost of (8) is a rank of an $n \times 2|A|$ matrix whose nonzero density is the stabilizer weight; sparse (area-law) tableaux make all rank-based observables, and the measurement updates that produce them, cheap.

8.2 Mutual and tripartite mutual information

For disjoint A, B ,

$$I(A : B) = S(A) + S(B) - S(A \cup B), \quad (9)$$

and for pairwise-disjoint A, B, C the tripartite mutual information is

$$I_3(A : B : C) = I(A : B) + I(A : C) - I(A : B \cup C) \in \mathbb{Z}, \quad (10)$$

a canonical MIPT order parameter (negative in the volume-law phase, near zero in the area-law phase) [5].

8.3 Code dimension

For a state on $2m$ qubits prepared as a Bell purification of m system qubits (Section 10), the *code dimension* is the system-half entanglement entropy

$$k \equiv S(\text{system}) = \text{rank}_{\mathbb{F}_2}(G|_{\{0, \dots, m-1\}}) - m \in [0, m], \quad (11)$$

the number of system qubits still entangled with the reference. It is the Gullans–Huse purification order parameter: $k = m$ for a fresh purification, $k \rightarrow 0$ as monitored dynamics purifies the system.

8.4 Weight spectrum

The *weight* of a generator is its number of non-identity Paulis, $\text{wt}(r) = \text{supp_len}[r]$. `generator_weights` returns the per-row weights; `stabilizer_weight_spectrum` the histogram over stabilizer rows; `average_stabilizer_weight` the mean. These quantify tableau sparsity (and hence simulation cost) directly.

Part II

The Circuits Layer

9 Graph-defined monitored circuits

The circuits layer builds random-Clifford+measurement circuits whose two-qubit gates are placed on the edges of a fixed graph. One configuration plus a sample seed determines a circuit realization reproducibly.

9.1 Graphs, matchings, and 1-factorizations

Definition 9.1. A *matching* of a graph $G = (V, E)$ is an edge set no two of which share a vertex; it is *perfect* if it covers every vertex. A *1-factorization* is a partition of E into perfect matchings.

By Vizing’s theorem the chromatic index of a simple graph is Δ or $\Delta + 1$; a Δ -regular graph is 1-factorable iff it is Class 1, and then its Δ color classes are perfect matchings. Six string-specified families are implemented: cycle, complete, path, a square two-dimensional open lattice, Newman–Watts shortcut graphs, and Watts–Strogatz rewired circulants. The last two accept parameters in the specification string and are realized from the configuration’s base seed. Arbitrary simple undirected NetworkX graphs are accepted through `from_networkx`. Two useful exact constructions (for even $|V| = n$) are:

- **Cycle C_n :** 2-regular, $\chi' = 2$, with two alternating perfect matchings $\{(0, 1), (2, 3), \dots\}$ and $\{(1, 2), (3, 4), \dots, (n - 1, 0)\}$.
- **Complete K_n :** $(n - 1)$ -regular, $\chi' = n - 1$, from the round-robin tournament 1-factorization (fix vertex $n - 1$, rotate the rest).

For odd n neither cycle nor complete has a perfect matching. Paths and open lattices carry proper edge-color classes, while stochastic and arbitrary graphs may carry only their edge list and, when available, a fresh perfect-matching sampler. A configuration rejects a matching mode that its realized graph cannot supply. Every topology stores a canonical edge list and graph6 string, together with any available factorization and matching sampler.

9.2 Gating

Each layer fires a set of gate edges, each carrying an independent uniform $\text{Sp}(4, \mathbb{F}_2)$ Clifford (Section 7.4):

- **Brickwork:** one disjoint stored matching or edge-color class. On even cycles and complete graphs these are perfect matchings ($n/2$ gates); path, lattice, and arbitrary-graph color classes may leave vertices idle. The matching is chosen by the matching mode: *round_robin* (cycle through the 1-factorization, layer t uses matching $t \bmod \chi'$), *palette* (uniform over the χ' stored classes), or *fresh* (uniform over *all* perfect matchings of the graph, when a sampler is available).
- **random_edge:** m distinct uniformly-random edges per layer (which may share vertices, so not a matching), where $m = \text{gates_per_layer}$ may be a constant or a function of n (e.g. $m = n/2$).
- **random_pool:** m uniformly-random edges sampled with replacement. Its default is $m = n/2$; repeated edges and shared vertices are allowed.
- **all_edges:** every stored graph edge fires once in deterministic edge-list order. Randomness is spent only on the Clifford choice and the measurement process.

9.3 Measurement process: candidates and firing

Each measured layer selects a *candidate* set $\mathcal{K}$ of qubits, then fires (measures in Z) each candidate independently with probability p ; the fired subset $\mathcal{F} \subseteq \mathcal{K}$ is projected. The candidate set is mode-specific:

- **bernoulli:** $\mathcal{K} = \{0, \dots, n - 1\}$ (every qubit).
- **gated:** $\mathcal{K} = \{\text{qubits the layer's gates touched}\}$.
- **random_pair:** $\mathcal{K} = \{2 \text{ qubits chosen uniformly without replacement}\}$.
- **uniform_count:** $\mathcal{K}$ is a uniformly sampled set of exactly `meas_count` distinct qubits (default $\max(1, \lfloor n/2 \rfloor)$).

Only $\mathcal{F}$ is measured; the unfired candidates are recorded (for inspection) but leave the state unchanged.

9.4 Depth normalization

Depth is measured in *gates per qubit*, so circuits are comparable across gating modes. The base budget is $T_{\text{base}} = \text{depth_factor} \cdot n$ for $O(n)$ and `until_purified` depth, or $\text{depth_factor} \cdot \lceil \log_2 n \rceil$ for $O(\log n)$. In `until_purified` mode this value is a cap: the runner checks the picture's order parameter adaptively and stops at its exact first zero. The normalization is calibrated to a full brickwork matching of $n/2$ gates that touches every qubit once, so T_{base} is its gates-per-qubit budget. A non-perfect stored color class still counts as one brickwork layer and can apply fewer gates; the record reports the realized total. A `random_edge` layer fires m gates touching $2m$ qubit-slots, i.e. a fraction $2m/n$ of the qubits per layer, so it needs

$$T_{\text{random_edge}} = \text{round}\left(T_{\text{base}} \cdot \frac{n}{2m}\right) \quad (12)$$

layers for the same budget; the same normalization applies to `random_pool`. Thus single-edge `random_edge` ($m = 1$) runs $O(n^2)$ layers, while $m = n/2$ matches brickwork. The `all_edges` architecture instead defines one layer as one pass over the complete stored edge list and therefore uses the base count. Finally, a positive `total_layers_override` bypasses every depth formula and specifies a literal measured-layer cap.

9.5 Expected gate-to-measurement ratio

With e_g the configured pre-run gate estimate ($e_g = n/2$ for brickwork, $e_g = m$ for the two random-edge modes, and $e_g = |E|$ for `all_edges`) and e_m the corresponding measurement estimate, the ratio recorded before simulation is $r = e_g/e_m$ (or ∞ when $e_m = 0$), where by measurement mode

$$e_m = \begin{cases} np & \text{bernoulli,} \\ 2e_g p & \text{gated (non-all_edges),} \\ |\bigcup_{e \in E} e| p & \text{gated (all_edges),} \\ 2p & \text{random_pair,} \\ \text{meas_count } p & \text{uniform_count.} \end{cases} \quad (13)$$

For the full-brickwork+bernoulli baseline this collapses to $r = \frac{n/2}{np} = \frac{1}{2p}$. For a non-perfect brickwork color class the configured $e_g = n/2$ is likewise a convention rather than the realized count. The non-all_edges gated estimate uses $2e_g$ even when gate endpoints overlap and is then a lower bound on the realized ratio. The record's realized gate and measurement totals, and hence its realized ratio, are exact.

9.6 Time series and checkpoints

The runner may record the order parameter after every measured layer. It also accepts selected one-based checkpoint layers. With no callback, each checkpoint stores an independent symplectic-tableau snapshot; with a callback, its read-only return value is stored instead. A callback may return the distinguished `CHECKPOINT_STOP` sentinel, which stops after that checkpoint without storing the sentinel. Post-loop observables are evaluated on the actual final state in all cases.

10 Physics pictures and order parameters

A *picture* fixes the initial state, the system/reference partition, and the order parameter. All order parameters are FCYBC ranks (Theorem 8.1).

- **pure_state:** n qubits in $|0^n\rangle$; order parameter is the half-cut entropy $S(\{0, \dots, n/2 - 1\})$ (volume-law vs. area-law).
- **purification:** $2n$ qubits with a Bell pair per system qubit $(H_i; CX_{i,i+n})$; order parameter is the code dimension $k = S(\text{system})$. Fresh: $k = n$; under monitoring $k \rightarrow 0$ (the Gullans–Huse purification transition).
- **single_ref:** $n+1$ qubits with one reference Bell-paired with system qubit $n-1$ $(H_{n-1}; CX_{n-1,n})$; order parameter is $S(\text{reference}) \in \{0, 1\}$.

Remark 10.1 (Measurement-induced phase transition). Random Clifford gates spread entanglement ballistically (volume law) while projective measurements destroy it. As the rate p crosses a critical p_c , the steady-state entanglement of the monitored-trajectory ensemble switches from volume law to area law: a measurement-induced phase transition. The pictures above expose its order parameters; locating p_c by finite-size scaling is the task of the analysis layer (Section 18).

11 Reproducibility

A realization is determined by `(base_seed, sample_seed)` via three independent pseudorandom streams in reproducibility schema v2. The construction stream is seeded by the ordered entropy array `[base_seed, sample_seed]` and fixes gate placement, Clifford indices, and measured-qubit selection in a fixed per-layer order. The measurement-outcome and global-scramble streams append the integer tags `0x6D656173` (“meas”) and `0x73637262` (“scrb”), respectively. Pair-keying prevents distinct cells with equal `base_seed + sample_seed` from sharing a stream. A stochastic string graph specification is realized separately from `base_seed` alone, so its geometry is quenched across the sample seeds of one configuration.

Part III

Implementation status and remaining roadmap

This section distinguishes capabilities already present in the package from remaining extensions.

12 Bravyi–Maslov $O(n^2)$ sampling

Implemented by `random_symplectic(..., sampler="bravyi_maslov")` and used by the default `auto` mode for $n \neq 2$. It samples the canonical decomposition $C = F_1 H S F_2$ with a Mallows-distributed permutation. Packed matrix assembly remains a possible optimization.

13 Koenig–Smolin integer indexing

Deterministic bijection between $[0, |\text{Sp}(2n, \mathbb{F}_2)|)$ and group elements (forward and inverse), via symplectic transvections, remains unimplemented.

14 Batched simulation

A `BatchedSparseGF2` carrying many trajectories in shared storage, for vectorized sweeps, remains a future extension. The circuit runner does batch the ordered primitive gates and measurements of each individual trajectory to reduce Python-to-Numba transition overhead.

15 Additional graph families

`path`, `2D lattice`, `from_networkx`, and `Newman–Watts` adapters; each supplies edges and (optionally) a 1-factorization and a perfect-matching sampler. These families are implemented in `circuits/graphs.py`, alongside `cycle`, `complete`, `Watts–Strogatz`, and other stochastic families.

16 Additional measurement modes

`uniform_count`, which selects a fixed number of candidate qubits per layer before `Bernoulli( $p$ )` firing, is implemented together with `bernoulli`, `gated`, and `random_pair`.

17 Adaptive depth and time series

`until_purified` early termination (stop when the order parameter vanishes) and per-layer order-parameter recording are implemented. The runner also supports selected one-based depth checkpoints, either as independent tableau snapshots or read-only callback values. A callback may return `CHECKPOINT_STOP` for checkpoint-granular early stopping.

18 Analysis and finite-size scaling

A sweep driver over (n, p, seed) with data collection and finite-size scaling to locate the critical point p_c and extract critical exponents is implemented in `sparsegf2.analysis`. The package provides sweep/study orchestration, atomic persistence, offline reduction, plotting, and scaling collapse; project-specific research estimators remain in `studies/`.

19 Publication-quality circuit rendering

A `quantikz/LATEX` renderer over the existing circuit-trace intermediate representation is implemented in `circuits/draw.py`, with PDF/PNG compilation when the external TeX and rasterization tools are present.

20 Additional pictures, observables, and noise models

Reserved for future order parameters, multipartite measures, and noisy (non-unitary, non-projective) channels.

References

- [1] S. Aaronson and D. Gottesman, *Improved Simulation of Stabilizer Circuits*, Phys. Rev. A **70**, 052328 (2004), [arXiv:quant-ph/0406196](https://arxiv.org/abs/quant-ph/0406196).

- [2] D. Fattal, T. S. Cubitt, Y. Yamamoto, S. Bravyi, I. L. Chuang, *Entanglement in the Stabilizer Formalism* (2004), [arXiv:quant-ph/0406168](#).
- [3] S. Bravyi and D. Maslov, *Hadamard-free circuits expose the structure of the Clifford group* (2020), [arXiv:2003.09412](#).
- [4] R. Koenig and J. A. Smolin, *How to efficiently select an arbitrary Clifford group element* (2014), [arXiv:1406.2170](#).
- [5] M. J. Gullans and D. A. Huse, *Dynamical Purification Phase Transition Induced by Quantum Measurements*, Phys. Rev. X **10**, 041020 (2020), [arXiv:1905.05195](#).